

# Measuring the Distance of Programming Languages Using Fixed-Points 후속 연구 진행 상황

한상곤 (부산대학교 대학원 정보융합공학과)

# 1. 연구 개요: 반복 번역에서 무엇을 측정할까?

다른 언어로 옮겼다가 돌아오면, 코드와 기능은 얼마나 유지될까?

- 실험 방법: 언어 간 왕복 번역 반복 (예: C++ → Python → C++)
- 측정 항목
  - 코드가 안정화될 때까지의 왕복 횟수
  - 원본 코드와의 구조적 유사도 (MOSS 등)
  - 테스트 통과 (기능 유지) 여부
- 연구 목표: 언어, 모델, 번역 지시문, 문제에 따른 결과의 차이와 신뢰도 제시

## 2. 관련 연구 (1/4): 되돌려서 번역 결과 평가하기

### 왕복 정합성(Round-Trip Correctness, RTC) 평가

- 개념: 변환 결과를 다시 원래 형태로 되돌려 원본과 비교하는 평가 방법 (Allamanis 등)
  - 예시: 코드 → 자연어 설명 → 코드로 변환한 뒤, 복원된 코드가 원래 기능을 수행하는지 확인
- 특징 및 한계
  - 기존 성능 평가 점수와 높은 상관관계를 보임
  - 단, 테스트 통과는 사용된 입력에 대한 올바른 동작을 의미할 뿐, 모든 상황에서의 정답을 항상 보장하지는 않음
- 본 연구의 확장 아이디어
  - 안정화까지 걸리는 왕복 횟수 측정
  - 중간 단계 오류 분석 및 다양한 모델 적용 결과 비교

---

Allamanis, Panthaplackel, Yin (2024), [Unsupervised Evaluation of Code LLMs with Round-Trip Correctness](#), ICML, PMLR 235:1050-1066.

## 2. 관련 연구 (2/4): 사람의 언어를 비교하는 방법

자연어 비교 방법을 프로그래밍 언어에 적용할 수 있을까?

- 기존 자연어 비교 연구
  - Petroni·Serva: 같은 뜻의 단어끼리 철자가 얼마나 다른지 계산하고 평균 내어 언어 비교
  - DistaLs: 언어의 기원, 문법 유형 등 다각적 기준으로 언어 간 차이를 도출
- 본 연구의 접근 (프로그래밍 언어)
  - 측정 기준: 코드 유사도, 테스트 통과율, 반복 횟수를 각각 측정
  - 비교 통제: 동일한 문제들을 번역해야 언어별 결과를 공정하게 비교 가능
- 주의점: 실험 결과로 도출된 비슷한 언어들의 군집은 관측된 유사성을 보여줄 뿐, 프로그래밍 언어들의 역사적 파생 관계를 직접 입증하는 것은 아님

---

Petroni·Serva (2010), [Measures of lexical distance between languages](#). van der Goot 등 (2025), [DistaLs](#).

## 2. 관련 연구 (3/4): 코드 유사도

### 코드가 얼마나 비슷한지 측정하는 도구들

| 측정 방법            | 비교 대상                       | 고려 사항                    |
|------------------|-----------------------------|--------------------------|
| MOSS / Winnowing | 코드에서 서로 겹치는 부분              | 비슷한 코드라도 기능은 다를 수 있음     |
| TSED             | 문법 구조 기반의 트리 (AST)          | 분석 도구 및 구조 변경 비용 설정에 의존적 |
| CSSG             | 실행 흐름 및 데이터 의존 관계 그래프 (PDG) | 추출 한계와 분석 도구에 따라 편차 발생   |

- 비교 원칙: 원본 코드와 왕복 변환된 코드를 동일 언어 내에서 비교  
(다른 언어의 구조를 직접 비교하려면 복잡한 공통 표현과 대응 규칙이 필요)
- 실험 적용: 기능 유지 여부는 별도의 실행 테스트로 분리하여 확인

---

[MOSS 공식 설명](#), [Winnowing 원문](#), [TSED 논문](#), [CSSG v2](#).

## 2. 관련 연구 (4/4): 이 연구에서 말하는 거리

### 정보 이론의 '거리'와 본 연구의 '실험적 지표'

- 정보 거리 (NID): 한 데이터로 다른 데이터를 설명하는 데 필요한 정보량을 의미하는 이론적 기준
  - 모델 호출 횟수를 세는 것만으로 이 정보 거리를 계산했다고 볼 수는 없음
- 본 연구의 "거리" (Distance)
  - 정해진 조건 하에서 왕복 번역이 얼마나 잘 유지되는지를 나타내는 실험적 지표
- 후속 연구의 목표
  - 단순 실패와 코드 안정화를 명확히 구분
  - 제안된 여러 지표가 번역 오류를 얼마나 잘 설명하는지 분석
  - 모델이나 문제를 바꾸더라도 결과가 일관되게 유지되는지 검증

---

Li, Chen, Li, Ma, Vitányi (2004), [The Similarity Metric](#), IEEE Transactions on Information Theory 50(12):3250-3264.

## 측정 지표: 성공률, 반복 횟수, 코드 변화

1. 언어간 차이 지표: 언어 간 차이 지표 ( $d_{SF}$ ): 100건 중 80건 성공 시 차이 지표는 0.2. 값이 작을수록 성공률이 높고 서로 간의 차이가 적음을 의미

$$d_{SF} = 1 - p_{SF}$$

2. 성공률: 2회 변환에서도 코드가 유지되며 모든 단계에서 테스트 통과한 경우

$$p_{SF} = (\text{성공한실험수})/(\text{평가가능한전체실험수})$$

| 지표                              | 설명                                       |
|---------------------------------|------------------------------------------|
| 안정화(테스트 통과)까지의 왕복 횟수 ( $\tau$ ) | 성공한 실험의 중앙값, IQR(가운데 50% 범위), 실험 총 수 보고  |
| 원본 대비 코드 변화 ( $\Delta_0$ )      | 성공한 실험에서 최초 원본과 최종 코드 간의 구조적 차이 비교       |
| 실패 이유별 비율                       | 기능 오류, 무한 반복 패턴, 최대 허용 횟수 도달, 추가 확인 실패 등 |